

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN - ĐIỆN TỬ
BỘ MÔN KỸ THUẬT MÁY TÍNH



MÔN HỌC: BLOCK CHAIN

**ĐỀ TÀI: HỘP ĐEN AN TOÀN VÀ ĐỊNH DANH CHO
ROBOT HỢP TÁC (HRC SAFETY LOG)**

GVHD : TS. Huỳnh Thế Thiện
Mã môn học : BCAP437964_01
Ngành : Công Nghệ Kỹ Thuật Máy Tính
Khoá : 2023
Danh sách thành viên :

Họ và tên	MSSV	Hoàn Thành
Lê Văn Lương (Nhóm trưởng)	23119166	100%
Đoàn Nguyễn Hoàng Minh	23119168	100%
Nguyễn Ngọc Tú	23119224	100%
Đinh Thiện Vinh	23119226	100%
Nguyễn Hồ Minh Vy	23119230	100%

LỜI CẢM ƠN

Trong quá trình học tập và thực hiện báo cáo cuối kỳ của môn học Blockchain và Ứng dụng, nhóm chúng em đã nhận được rất nhiều sự quan tâm, hướng dẫn và hỗ trợ tận tình từ Thầy cùng các bạn. Những sự giúp đỡ đó đã tạo điều kiện thuận lợi để nhóm có thể hoàn thành tốt quá trình học tập và thực hiện đề tài.

Nhóm chúng em xin bày tỏ lòng biết ơn sâu sắc đến Thầy TS. Huỳnh Thế Thiện, giảng viên hướng dẫn học phần, đã tận tình giảng dạy, định hướng tư duy và chia sẻ những lời góp ý chân thành trong suốt quá trình học tập và thực hiện đề án. Những kiến thức Thầy truyền đạt về công nghệ Blockchain, hợp đồng thông minh và các ứng dụng thực tiễn không chỉ giúp nhóm hình thành tư duy kỹ thuật vững chắc, mà còn là nền tảng để chúng em xây dựng thành công hệ thống HRC Safety Log - hộp đen an toàn và định danh cho robot hợp tác.

Nhóm cũng xin chân thành cảm ơn các bạn trong lớp đã hỗ trợ, trao đổi kinh nghiệm và đóng góp ý kiến trong quá trình hoàn thiện đề tài. Sự hợp tác và chia sẻ đó đã giúp nhóm vượt qua nhiều khó khăn khi triển khai hệ thống tích hợp IoT - AI - Blockchain, góp phần vào kết quả của bài báo cáo.

Mặc dù nhóm đã cố gắng hoàn thành đề án một cách tốt nhất, nhưng do kinh nghiệm còn hạn chế và đây là lần đầu tiếp cận công nghệ Blockchain ở mức độ ứng dụng thực tế, báo cáo khó tránh khỏi những thiếu sót. Nhóm rất mong nhận được sự góp ý, nhận xét từ Thầy để có thể tiếp tục hoàn thiện kiến thức và phát triển bài báo cáo tốt hơn trong tương lai.

Nhóm xin chân thành cảm ơn!

Nhóm thực hiện đề tài
(Ký và ghi rõ họ tên)

TÓM TẮT

Trong lĩnh vực Blockchain và Hệ thống Nhúng, việc tích hợp công nghệ chuỗi khối vào các hệ thống vật lý thực tế đang mở ra hướng ứng dụng mới mẻ và đầy tiềm năng. Một trong những thách thức lớn của môi trường sản xuất hiện đại là đảm bảo an toàn lao động khi con người làm việc chung không gian với robot hợp tác (cobot). Việc xây dựng một hệ thống ghi nhật ký an toàn có tính bất biến, minh bạch và có thể kiểm chứng là bài toán thiết thực mà đề án hướng đến giải quyết. Do đó, đề tài tập trung nghiên cứu và hiện thực hóa hệ thống HRC Safety Log - hộp đen an toàn và định danh cho robot hợp tác - trên nền tảng tích hợp IoT - AI - Blockchain.

Hệ thống được thiết kế theo kiến trúc CPS ba tầng. Tầng IoT sử dụng vi điều khiển ESP32 kết hợp cảm biến khoảng cách VL53L0X, áp dụng thuật toán Speed & Separation Monitoring để tự ngắt relay bảo vệ trong vòng mili-giây mà không phụ thuộc mạng. Tầng AI triển khai tổ hợp mô hình RandomForest, IsolationForest và Autoencoder nhằm phân biệt người thật với cảm biến báo sai, đạt precision 1,000 và recall 0,996. Tầng Blockchain sử dụng smart contract Solidity để lưu bằng chứng bất biến dưới dạng Merkle root của log vận hành, ghi nhận sự cố và phân quyền khóa/mở robot - trong đó chỉ duy nhất SUPERVISOR được quyền mở khóa.

Smart contract đạt coverage 100% với 50/50 test case. Hệ thống đã được kiểm thử end-to-end với 198 pytest và triển khai thành công trên cả mạng local lẫn testnet Sepolia, kèm giao diện Dashboard trực quan tích hợp MetaMask để SUPERVISOR ký mở khóa trực tiếp từ trình duyệt.

Mục lục

1	GIỚI THIỆU TỔNG QUAN	1
1.1	Giới thiệu bài toán	1
1.2	Mục tiêu đề tài	1
1.3	Phạm vi và giới hạn	1
2	CƠ SỞ LÝ THUYẾT	2
2.1	Tổng quan về CPS và Cobot (Collaborative robot)	2
2.2	SSM theo ISO/TS 15066	2
2.3	Blockchain, Smart Contract và cây Merkle	2
2.4	Isolation Forest	3
2.5	MQTT và HMAC	3
3	THIẾT KẾ HỆ THỐNG	5
3.1	Kiến trúc tổng thể	5
3.2	Tầng IoT	5
3.2.1	Vai trò	5
3.2.2	Phần cứng	6
3.2.3	Thuật toán SSM	6
3.2.4	Gói tin MQTT	7
3.3	Tầng AI	8
3.3.1	Vai trò	8
3.3.2	Tập dữ liệu	8
3.3.3	Kiến trúc mô hình	9
3.3.4	Triển khai	9
3.4	Tầng Blockchain	9
3.4.1	Vai trò	9
3.4.2	Phân quyền	10
3.4.3	Máy trạng thái và các hàm	10
3.4.4	Merkle tree và chống replay	11
3.5	Backend và Dashboard	12
3.5.1	Vai trò	12
3.5.2	Sơ đồ tuần tự xử lý sự cố	12

4 KẾT QUẢ	13
4.1 Môi trường và kịch bản	13
4.2 Kết quả IoT	13
4.3 Kết quả AI	14
4.4 Kết quả Smart Contract	15
4.5 Kết quả end-to-end	17
5 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	18
5.1 Kết luận	18
5.2 Hướng phát triển	18

Danh sách hình vẽ

2.1	Cấu trúc cây Merkle	3
3.1	Sơ đồ kiến trúc tổng thể hệ thống	5
3.2	Sơ đồ kết nối phần cứng	6
4.1	Ma trận nhầm lẫn của Random Forest	14
4.2	Phân bố điểm bất thường của dữ liệu bình thường và bất thường	15

Danh sách bảng

3.1	Tham số SSM trong quá trình demo	7
3.2	Các topic MQTT	8
3.3	Phân bố tập dữ liệu (chu kỳ 100 ms/bản ghi)	9
3.4	Các vai trò trong contract	10
3.5	Các hàm chính của HRCSafetyLog	11
3.6	Trình tự xử lý một sự cố	12
4.1	Môi trường thực nghiệm	13
4.2	Kết quả tầng IoT	13
4.3	Kết quả phân loại của Random Forest (không tính lỗi trôi dạt)	14
4.4	Kết quả phát hiện bất thường	14
4.5	Xác nhận sự cố vùng DANGER	15
4.6	Chi phí gas (solc 0.8.28, optimizer 200 runs)	16
4.7	Chi phí một ca 8 giờ	16
4.8	Kết quả demo trên mạch thật	17

CHƯƠNG 1

GIỚI THIỆU TỔNG QUAN

1.1 Giới thiệu bài toán

Trong các nhà máy hiện nay, robot công nghiệp không chỉ làm việc trong những khu vực riêng biệt với con người mà còn làm và hoạt động chung khu vực với con người. Robot hợp tác (collaborative robot) được thiết kế để làm việc ngay bên cạnh công nhân và dùng chung không gian làm việc. Cách làm này giúp dây chuyền linh hoạt hơn, nhưng đồng thời cũng làm tăng nguy cơ va chạm giữa người và robot. Vì vậy, các tiêu chuẩn ISO 10218 và ISO/TS 15066 đã đưa ra những yêu cầu an toàn riêng cho robot hợp tác [2, 3].

Bên cạnh việc phòng tránh tai nạn, một vấn đề khác cũng rất quan trọng là truy vết sau sự cố. Khi xảy ra va chạm, nhà máy, hãng sản xuất robot và bên bảo hiểm đều cần dữ liệu vận hành để xác định nguyên nhân và trách nhiệm. Tuy nhiên, nhật ký (log) thường được lưu trên máy chủ của một bên nên có thể bị chỉnh sửa hoặc xóa do đó làm cho các bên khó tin tưởng lẫn nhau. Blockchain có đặc điểm là dữ liệu đã ghi thì không thể sửa mà không bị phát hiện, vì vậy có thể dùng để xây dựng một “hộp đen” cho robot, tương tự hộp đen trên máy bay [11].

Xuất phát từ hai vấn đề trên, nhóm chọn đề tài “**Hộp đen an toàn và định danh cho robot hợp tác (HRC Safety Log)**”, kết hợp ba công nghệ IoT, AI và Blockchain.

1.2 Mục tiêu đề tài

Xây dựng mô hình hệ thống thực - ảo gồm ba tầng, chạy thông suốt End-to-End:

- **IoT**: đo khoảng cách người - robot, tự dừng robot khi người vào vùng nguy hiểm.
- **AI**: phát hiện hành vi bất thường và xác nhận sự cố.
- **Blockchain**: lưu vết nhật ký không thể chỉnh sửa, tự khóa robot khi có sự cố, chỉ người có thẩm quyền mới được mở khóa.

1.3 Phạm vi và giới hạn

Hệ thống được thực hiện trên mô hình thu nhỏ: servo đóng vai trò tay robot, cảm biến siêu âm HC-SR04 thay cho máy quét laser công nghiệp. Smart Contract chỉ triển khai trên mạng Hardhat cục bộ. Dữ liệu AI do nhóm tự thu thập và sinh bằng giả lập. Hệ thống mang tính minh họa, chưa đạt chứng nhận an toàn chức năng.

CHƯƠNG 2

CƠ SỞ LÝ THUYẾT

2.1 Tổng quan về CPS và Cobot (Collaborative robot)

Hệ thống thực - ảo (Cyber Physical System: CPS) là hệ thống kết hợp chặt chẽ giữa phần tính toán, mạng truyền thông và quá trình vật lý. Cảm biến thu thập dữ liệu từ thế giới thực, phần mềm xử lý dữ liệu, sau đó hệ thống tác động ngược lại thế giới thực. Trong đề tài này, ESP32 cùng cảm biến và relay là phần vật lý, còn backend, mô hình AI và blockchain là phần ảo.

Cộng tác người - robot (Human-Robot Collaboration: HRC) là hình thức người và robot cùng làm việc trong một không gian chung. Tiêu chuẩn ISO 10218-1 và ISO/TS 15066 chia hoạt động cộng tác thành bốn hình thức: dừng có giám sát an toàn, dẫn hướng bằng tay, giám sát tốc độ và khoảng cách (SSM), và giới hạn công suất, lực [2, 3]. Nhóm chọn hình thức SSM vì có thể thực hiện bằng cảm biến khoảng cách, không cần robot có cảm biến lực đắt tiền.

2.2 SSM theo ISO/TS 15066

Nguyên lý của SSM là robot và người phải luôn giữ một khoảng cách an toàn tối thiểu. Nếu khoảng cách thực tế nhỏ hơn khoảng cách này thì robot phải dừng lại [3, 4]. Khi giả sử tốc độ của người và robot không đổi, khoảng cách bảo vệ tối thiểu S có thể viết gọn như sau [4]:

$$S = v_H \cdot (T_R + T_S) + v_R \cdot T_R + S_S + C + Z_R + Z_S \quad (2.1)$$

Trong đó: v_H là tốc độ người tiến lại gần robot; v_R là tốc độ robot di chuyển về phía người; T_R là thời gian phản ứng của hệ thống; T_S là thời gian để robot dừng hẳn; S_S là quãng đường robot đi được trong lúc dừng; C là khoảng xâm nhập (phần tay người có thể vươn tới); Z_R và Z_S lần lượt là sai số vị trí của robot và của cảm biến.

Nguyên tắc hoạt động: nếu khoảng cách đo được d nhỏ hơn S thì robot dừng; khi d lớn hơn hoặc bằng S thì robot được phép chạy lại. Từ công thức có thể thấy robot chạy càng nhanh thì S càng lớn, nghĩa là cần giữ khoảng cách xa hơn.

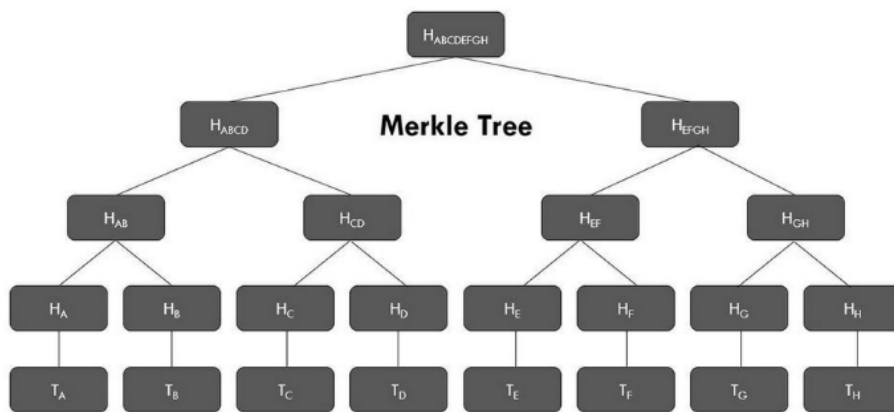
2.3 Blockchain, Smart Contract và cây Merkle

Blockchain là một sổ cái phân tán, trong đó dữ liệu được gom thành các khối (block). Mỗi khối chứa giá trị băm (hash) của khối trước nên các khối liên kết với nhau thành chuỗi. Nếu ai đó sửa dữ liệu trong một khối, giá trị băm sẽ thay đổi và phá vỡ liên kết với các khối phía sau, nên việc sửa sẽ bị các nút khác trong mạng phát hiện [5].

Hàm băm biến một dữ liệu bất kỳ thành một chuỗi có độ dài cố định. Ethereum sử dụng hàm Keccak-256 cho ra kết quả 32 byte. Chỉ cần thay đổi một bit của dữ liệu đầu vào, kết quả băm sẽ khác hoàn toàn.

Smart Contract là chương trình được lưu và chạy trên blockchain, tự động thực thi khi có giao dịch gọi đến và không thể bị sửa sau khi đã triển khai. Trên Ethereum, Smart Contract thường được viết bằng ngôn ngữ Solidity. Mỗi giao dịch đều được ký bằng khóa riêng (private key) của người gửi, nhờ đó hệ thống biết chính xác ai đã thực hiện hành động nào. Mỗi giao dịch cũng tốn phí gas và việc ghi dữ liệu lên chuỗi khá tốn kém, vì vậy thực tế chỉ nên lưu giá trị băm thay vì dữ liệu gốc.

Cây Merkle là cây nhị phân dùng để đại diện cho một tập dữ liệu lớn bằng một giá trị băm duy nhất [7]. Mỗi lá là giá trị băm của một bản ghi, mỗi nút cha là giá trị băm của hai nút con và nút gốc được gọi là Merkle root (Hình 2.1). Vì vậy việc lưu Merkle root (32 byte) lên blockchain có thể đại diện cho tất cả bản ghi; nếu một bản ghi bị sửa thì Merkle root sẽ thay đổi. Để chứng minh một bản ghi thuộc lô đó, chỉ cần khoảng $\log_2(n)$ giá trị băm (gọi là Merkle proof) thay vì toàn bộ n bản ghi.



Hình 2.1: Cấu trúc cây Merkle

2.4 Isolation Forest

Isolation Forest là thuật toán phát hiện bất thường không giám sát, được Liu và cộng sự đề xuất năm 2008 [8]. Ý tưởng chính là các điểm bất thường “ít và khác biệt” so với phần còn lại, nên dễ bị tách riêng hơn.

Thuật toán xây dựng nhiều cây ngẫu nhiên. Ở mỗi nút, cây chọn ngẫu nhiên một đặc trưng và một ngưỡng để chia dữ liệu làm hai. Điểm bất thường thường bị tách riêng chỉ sau vài lần chia (đường đi từ gốc đến lá ngắn), trong khi điểm bình thường cần nhiều lần chia hơn. Điểm bất thường được tính dựa trên độ dài đường đi trung bình qua tất cả các cây: giá trị càng gần 1 thì mẫu càng có khả năng bất thường.

Thuật toán này chạy nhanh, nhẹ và không cần nhiều dữ liệu có nhãn. Điều này phù hợp với đề tài vì dữ liệu sự cố thực tế rất ít.

2.5 MQTT và HMAC

MQTT là giao thức truyền tin nhẹ theo mô hình publish/subscribe, được chuẩn hóa bởi tổ chức OASIS [9]. Hệ thống MQTT gồm bên gửi (publisher), bên nhận (subscriber) và máy chủ trung gian (broker). Bên gửi đăng tin lên một chủ đề (topic), broker sẽ chuyển tin đến tất cả bên nhận đã đăng ký

chủ đề đó. MQTT có gói tin nhỏ, tốn ít tài nguyên nên rất phù hợp với vi điều khiển như ESP32.

HMAC (Hash-based Message Authentication Code) là mã xác thực thông điệp, được tạo bằng hàm băm kết hợp với một khóa bí mật mà hai bên cùng biết [10]. Bên gửi tính HMAC của bản tin và gửi kèm; bên nhận tính lại HMAC và so sánh. Nếu hai giá trị khớp thì bản tin đúng nguồn và không bị sửa trên đường truyền. Nhóm sử dụng HMAC-SHA256, kết hợp với số thứ tự (seq) tăng dần trong mỗi bản tin để chống việc phát lại (replay), hay là kẻ gian gửi lại một bản tin cũ hợp lệ.

CHƯƠNG 3

THIẾT KẾ HỆ THỐNG

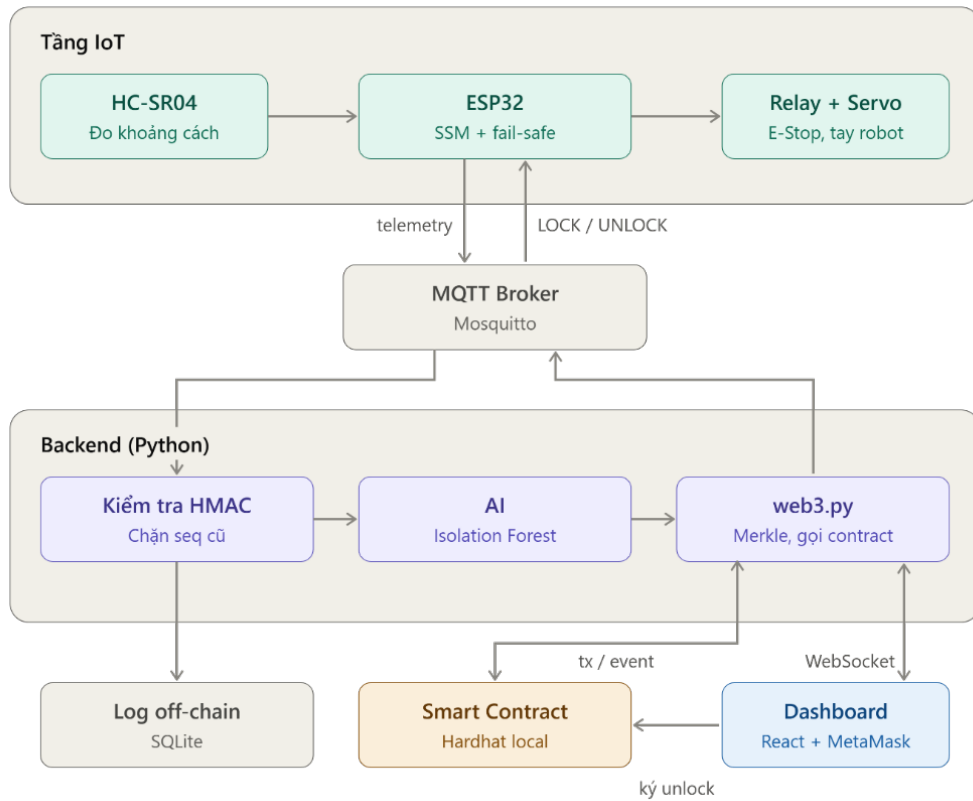
3.1 Kiến trúc tổng thể

Hệ thống gồm ba tầng IoT, AI và Blockchain, được tổ chức thành hai luồng xử lý song song (Hình 3.1).

Luồng an toàn (fast lane): ESP32 tự tính khoảng cách an toàn và ngắt relay ngay tại chỗ khi người đi vào vùng nguy hiểm. Luồng này không phụ thuộc vào mạng hay blockchain, nên robot dừng được trong thời gian rất ngắn.

Luồng truy vết (trust lane): dữ liệu được gửi qua MQTT về backend. Mô hình AI phân tích, sau đó Smart Contract ghi lại bằng chứng, khóa robot và lưu vết người chịu trách nhiệm.

Blockchain có độ trễ cỡ giây, nên nhóm không đưa blockchain vào vòng điều khiển an toàn. Blockchain chỉ đóng vai trò “hộp đen” và “trọng tài” của hệ thống.



Hình 3.1: Sơ đồ kiến trúc tổng thể hệ thống

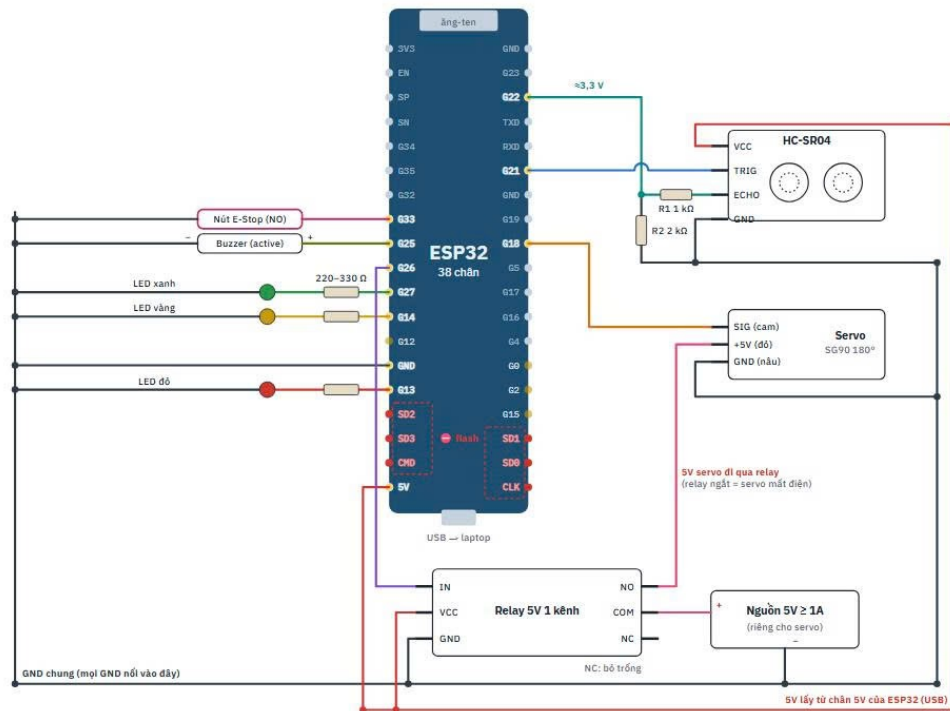
3.2 Tầng IoT

3.2.1 Vai trò

Tầng IoT thu thập khoảng cách giữa người và robot, tính khoảng cách an toàn theo SSM, dừng robot khi cần và gửi dữ liệu vận hành về backend.

3.2.2 Phần cứng

- **ESP32**: vi điều khiển trung tâm, có sẵn WiFi để gửi dữ liệu MQTT.
- **Cảm biến siêu âm HC-SR04**: đo khoảng cách từ 2 cm đến khoảng 4 m. Chân Echo xuất mức 5 V nên nhóm dùng cầu chia áp [1 kΩ - 2 kΩ] để hạ xuống 3,3 V trước khi đưa vào ESP32.
- **Servo SG90**: mô phỏng tay robot. Tốc độ quay được điều khiển để giả lập các mức tốc độ robot khác nhau.
- **Relay và buzzer**: đóng vai trò nút dừng khẩn cấp (E-Stop). Relay ngắt nguồn servo, buzzer báo động.
- **LED trạng thái**: xanh (SAFE), vàng (WARNING), đỏ (DANGER/LOCKED).



Hình 3.2: Sơ đồ kết nối phần cứng

3.2.3 Thuật toán SSM

HC-SR04 đo độ rộng t của xung ECHO. Khoảng cách được tính theo

$$d = \frac{t \times v_{\text{âm}}}{2}, \quad \text{với } v_{\text{âm}} \approx 343 \text{ m/s ở } 20^\circ\text{C}, \quad (3.1)$$

kết quả làm tròn xuống để khoảng cách báo ra luôn hơi gần hơn thực tế. Theo khuyến nghị của nhà sản xuất, mỗi lần đo cách nhau ít nhất 60 ms để sóng dội của lần trước tắt hẳn, nên nhóm chọn chu kỳ đo 60 ms (khoảng 16,7 Hz). Khi không có vật trong tầm tin cậy (xa hơn 1.200 mm), cảm biến báo giá trị quy ước 8190 (“ngoài tầm”).

ESP32 tính khoảng cách bảo vệ S theo phương pháp SSM của ISO/TS 15066:

$$S = \left\lceil \frac{v_H \cdot (T_R + T_S) + v_R \cdot T_R}{1000} \right\rceil + S_S + C + Z_R + Z_S \quad (3.2)$$

Trong đó v_R là tốc độ đã ra lệnh cho servo ở chu kỳ trước, vì SG90 không có encoder để đo tốc độ thật. Kết quả được làm tròn lên. Bảng 3.1 là bộ tham số `demo_close` đang chạy trên mạch.

Bảng 3.1: Tham số SSM trong quá trình demo

Tham số	Giá trị	Ý nghĩa
v_H	50 mm/s	Tốc độ tay tiến lại gần (giả định cho demo)
T_R	150 ms	Thời gian phản ứng: chu kỳ 60 ms + chờ tiếng vọng + xử lý
T_S	100 ms	Thời gian relay nhả và servo dừng
S_S, C	0 mm	Quãng trượt khi dừng và khoảng xâm nhập (bỏ qua với mô hình nhỏ)
Z_R, Z_S	5 mm, 10 mm	Sai số vị trí servo và sai số cảm biến
Lề cảnh báo	50 mm	Độ rộng vùng WARNING
Tốc độ vận hành / giới hạn	150 / 250 mm/s	Vượt 250 mm/s thì ngắt (OVERSPEED)

Với tốc độ 150 mm/s ta có $S = 50$ mm; khi robot đã đứng yên S co lại còn 28 mm; ở 250 mm/s $S = 65$ mm. Các giá trị này chỉ để biểu diễn trên mô hình, để robot dừng khi tay cách cảm biến khoảng 5 cm. Nhóm không sử dụng bộ tham chiếu theo tiêu chuẩn ($v_H = 1.600$ mm/s, $T_S = 300$ ms, $C = 850$ mm...) vì S vượt xa tầm đo của mô hình. Dựa vào khoảng cách, hệ thống chia ba vùng:

- **DANGER** khi $d \leq S$: ngắt relay ngay, còi kêu, đèn đỏ.
- **WARNING** khi $S < d \leq S + 50$ mm: robot giảm tốc còn 50%, đèn vàng.
- **SAFE** khi $d > S + 50$ mm: robot chạy bình thường, đèn xanh.

3.2.4 Gói tin MQTT

ESP32 và backend trao đổi qua broker Mosquitto trên ba topic (Bảng 3.2). Nội dung là JSON, mọi gói đều có trường `hmac` là HMAC-SHA256 của một chuỗi chuẩn hóa, dùng khóa bí mật 32 byte dùng chung giữa ESP32 và backend.

Bảng 3.2: Các topic MQTT

Topic	Chiều	Nội dung chính	Tần suất
<code>hrc/1/telemetry</code>	ESP32 → backend	<code>robot_id</code> , <code>seq</code> , <code>ts_ms</code> , <code>distance_mm</code> , <code>v_robot_mmpps</code> , <code>zone</code> , <code>estop</code> , <code>hmac</code>	10 Hz
<code>hrc/1/heartbeat</code>	ESP32 → backend	<code>ts_ms</code> , <code>state</code> (khóa hay không), <code>estop</code> , <code>cfg</code> (mã cấu hình), <code>hmac</code>	2 Hz
<code>hrc/1/cmd</code>	backend → ESP32	<code>cmd</code> (LOCK/UNLOCK), <code>nonce</code> , <code>hmac</code> - QoS 1	Khi có sự kiện

Ví dụ, chuỗi được ký của một bản tin telemetry có dạng:

```
robot_id|seq|ts_ms|distance_mm|v_robot_mmpps|zone|estop
```

Để chống phát lại bản tin cũ (replay), mỗi telemetry có số thứ tự `seq` tăng dần; `seq` được lưu vào NVS theo khối 1.000 số nên không bị quay về 0 khi mất điện. Lệnh LOCK/UNLOCK có `nonce` là thời gian Unix (ms), phải lớn hơn lệnh trước và không lệch quá 30 giây so với đồng hồ ESP32. Vì vậy ESP32 phải đồng bộ giờ qua NTP trước khi gửi dữ liệu. Khi ESP32 mất kết nối đột ngột, broker tự phát tín hiệu (LWT) để backend biết thiết bị đã offline.

3.3 Tầng AI

3.3.1 Vai trò

Nếu chỉ dùng luật “cứ vào DANGER là sự cố” thì mỗi lần cảm biến báo sai (ví dụ một xung nhiễu tụt xuống vài cm) robot sẽ bị khóa oan và phải gọi supervisor tới mở. AI trong đề tài làm hai việc: phân biệt người thật với cảm biến lỗi trước khi ghi sự cố lên blockchain, và phát hiện dữ liệu bất thường. AI không nằm trong vòng dừng robot; việc dừng vẫn do fast lane làm.

3.3.2 Tập dữ liệu

Chưa có đủ dữ liệu thật, nên nhóm sinh dữ liệu có nhãn bằng simulator viết bằng Python, chạy đúng logic của firmware. Simulator có các kịch bản: làm việc bình thường, người tiến vào vùng nguy hiểm, robot vượt tốc và cảm biến lỗi (xung nhiễu, mất tín hiệu, trôi dần). Mỗi bản ghi có một trong bốn nhãn NORMAL, INTRUSION, OVERSPEED, SENSOR_FAULT. Dữ liệu được chia theo seed (mỗi seed là một lần chạy độc lập) chứ không chia ngẫu nhiên theo dòng, để cửa sổ của tập test không lọt vào tập train.

Bảng 3.3: Phân bố tập dữ liệu (chu kỳ 100 ms/bản ghi)

Tập	Số bản ghi	NORMAL	INTRUSION	OVERSPEED	SENSOR_FAULT
Train	180.000	144.177	5.809	10.200	19.814
Validation	54.000	43.419	1.743	3.060	5.778
Test	90.000	72.438	2.896	5.100	9.566

3.3.3 Kiến trúc mô hình

Mô hình không nhìn từng số đo lẻ mà nhìn cửa sổ 2 giây (20 bản ghi). Từ mỗi cửa sổ nhóm tính 20 đặc trưng như khoảng cách nhỏ nhất, tốc độ tiến lại gần, độ nhảy lớn nhất giữa hai mẫu, độ rung quanh đường hồi quy, tỉ lệ mẫu ngoài tầm... Người thật đi tới thì khoảng cách giảm đều, còn cảm biến lỗi thì nhảy vọt. Nhóm dùng ba mô hình:

- **Random Forest** (60 cây, sâu tối đa 12): phân loại bốn nhãn. Chỉ khi vùng là DANGER và Random Forest xác nhận INTRUSION thì backend mới ghi sự cố.
- **Isolation Forest** (100 cây): phát hiện bất thường, chỉ học trên dữ liệu bình thường.
- **Autoencoder** (20-16-8-4-8-16-20, 1.032 tham số): nén rồi dựng lại đặc trưng; dữ liệu lạ sẽ dựng lại sai nhiều. Ngưỡng của cả hai mô hình bất thường là phân vị 99% sai số trên dữ liệu bình thường của tập validation.

3.3.4 Triển khai

Mô hình sau khi huấn luyện được lưu thành file và nạp vào backend để suy luận theo thời gian thực. (Nếu nhóm làm phần sáng tạo:) Nhóm huấn luyện thêm một mạng autoencoder nhỏ, lượng tử hóa sang INT8 bằng TensorFlow Lite và nạp lên ESP32 bằng thư viện esp-tflite-micro, để phát hiện bất thường ngay tại thiết bị.

3.4 Tầng Blockchain

3.4.1 Vai trò

Smart contract **HRCSafetyLog** (Solidity 0.8.28, dùng thư viện OpenZeppelin) không dùng robot. Nó có hai vai trò chính là: **hộp đen** (lưu Merkle root của log để phát hiện nếu ai sửa log) và **trọng tài** (phân quyền, khóa/mở robot, ghi lại ai chịu trách nhiệm). Contract được deploy lên chuỗi Hardhat chạy trên máy tính cục bộ.

3.4.2 Phân quyền

Bảng 3.4: Các vai trò trong contract

Vai trò	Ai giữ	Được làm gì
DEFAULT_ADMIN	Ví deploy	Đăng ký robot, cấp/thu quyền, xóa số lần vi phạm
DEVICE	Ví của backend (thay mặt robot)	<code>anchorLog</code> , <code>reportIncident</code>
OPERATOR	Người vận hành	<code>checkIn</code> / <code>checkOut</code> (nhận và trả ca)
SUPERVISOR	Người giám sát an toàn	<code>unlock</code> - người duy nhất mở khóa được
AUDITOR	Kiểm toán viên	<code>recordAudit</code> - ghi kết quả đối soát log

Điểm chính là tách quyền: backend khóa được robot nhưng không mở được; admin cũng không mở được vì không có quyền SUPERVISOR. Ngoài ra nhóm ghi đề hàm `_grantRole` để admin không cấp lại quyền OPERATOR cho người đã bị khóa, trừ khi reset số vi phạm trước (việc reset cũng phát event nên để lại dấu vết).

3.4.3 Máy trạng thái và các hàm

Mỗi robot có hai trạng thái:

- ACTIVE và LOCKED. `reportIncident` chuyển ACTIVE $\rightarrow$ LOCKED, chỉ `unlock` của SUPERVISOR mới đưa LOCKED $\rightarrow$ ACTIVE.
- Khi robot đang LOCKED, operator không `checkIn` được.

Bảng 3.5: Các hàm chính của HRCSafetyLog

Hàm	Quyền	Chức năng
<code>anchorLog(robotId, batchId, root, fromSeq, toSeq)</code>	DEVICE của robot	Ghi Merkle root của một batch log
<code>reportIncident(robotId, evidence, severity, operator)</code>	DEVICE của robot	Ghi sự cố, khóa robot, cộng vi phạm cho operator; đủ 3 lần thì thu quyền OPERATOR
<code>unlock(robotId)</code>	SUPERVISOR	Mở khóa, xóa operator đang giữ ca
<code>checkIn / checkOut</code>	OPERATOR / người đang giữ ca	Nhận và trả ca vận hành
<code>recordAudit(robotId, batchId, passed, reportHash)</code>	AUDITOR	Ghi kết quả đối soát (chỉ phát event)
<code>verifyRecord(batchId, record, proof)</code>	Ai cũng gọi được (view)	Kiểm tra một dòng log có thuộc batch đã neo không

Mọi thao tác quan trọng đều phát event (`LogAnchored`, `EmergencyStop`, `Unlocked`, `OperatorBlocked`, `AuditRecorded`...). Backend nghe event để gửi lệnh xuống robot, còn dashboard đọc event để hiện lịch sử. Khi sai điều kiện, contract trả về custom error có tham số (ví dụ `RobotIsLocked`, `InvalidBatchId`) nên dễ biết sai ở đâu.

3.4.4 Merkle tree và chống replay

Ghi từng dòng log lên chuỗi là không khả thi (10 dòng mỗi giây). Backend gom tối đa 256 bản ghi hoặc 30 giây thành một batch. Mỗi bản ghi thành một lá:

```
leaf = keccak256(abi.encode(7 trường của bản ghi))
```

Bảy trường này đúng bằng các trường của telemetry: `robot_id`, `seq`, `ts_ms`, `distance_mm`, `v_robot_mmpps`, `zone`, `estop`. Thứ tự phải giống nhau ở contract (Solidity), backend (Python) và phần test (TypeScript), nếu không root sẽ không khớp.

Các lá được ghép từng cặp (hash nhỏ hơn đứng trước) cho tới khi còn một Merkle root; chỉ root 32 byte này được ghi lên chuỗi. Sửa bất kỳ dòng nào thì root thay đổi và không khớp với root trên chuỗi. Để chứng minh một dòng thuộc batch chỉ cần khoảng 8 hash “hàng xóm” (với 256 bản ghi). Hàm `verifyRecord` tự tính lá từ dữ liệu gốc nên tránh được kiểu tấn công đưa node trung gian vào giả làm lá (second-preimage).

Để chống ghi đè hoặc phát lại batch cũ, `anchorLog` yêu cầu `batchId` phải bằng batch trước + 1 và `fromSeq` phải lớn hơn `toSeq` của batch trước. Như vậy không ai chèn hay thay một batch đã neo được.

3.5 Backend và Dashboard

3.5.1 Vai trò

Backend viết bằng Python, gồm các khối:

- Nhận dữ liệu MQTT và kiểm tra HMAC, seq.
- Chạy mô hình AI.
- Gom lô và tính Merkle root.
- Gọi Smart Contract qua thư viện web3.py.
- Lắng nghe sự kiện on-chain để gửi lệnh LOCK/UNLOCK xuống ESP32.

Dashboard viết bằng React và ethers.js, kết nối ví MetaMask với mạng Hardhat cục bộ. Dashboard hiển thị vùng an toàn theo thời gian thực, lịch sử sự cố cùng mã giao dịch (tx hash) và số block, và có nút mở khóa để SUPERVISOR ký giao dịch.

3.5.2 Sơ đồ tuần tự xử lý sự cố

Bảng 3.6: Trình tự xử lý một sự cố

Bước	Thành phần	Việc xảy ra
1	HC-SR04 → ESP32	Đo được $d \leq S \rightarrow$ vùng DANGER
2	ESP32	Ngắt relay ngay, servo dừng, còi kêu (fast lane)
3	ESP32 → Mosquitto → Backend	Telemetry có ký; backend kiểm HMAC, seq rồi lưu log
4	Backend (AI)	Random Forest xác nhận INTRUSION trên cửa sổ 2 giây
5	Backend → Contract	reportIncident : ghi sự cố, robot chuyển LOCKED, phát event EmergencyStop
6	Contract → Backend → ESP32	Backend nghe EmergencyStop , gửi lệnh LOCK có ký; ESP32 lưu trạng thái khóa vào NVS
7	Dashboard	Đọc chuỗi, hiện LOCKED và dòng sự cố mới
8	SUPERVISOR (MetaMask) → Contract	Kiểm tra hiện trường, ký unlock ; contract phát event Unlocked
9	Contract → Backend → ESP32	Backend nghe Unlocked , gửi UNLOCK có ký
10	ESP32	Vùng SAFE liên tục 1 giây thì đóng relay, robot chạy lại

CHƯƠNG 4

KẾT QUẢ

4.1 Môi trường và kịch bản

Bảng 4.1: Môi trường thực nghiệm

Thành phần	Cấu hình
Máy tính	Laptop Windows 11, Python 3.11, Node.js 24
Blockchain	Hardhat 3.17 (chuỗi local, chain 31337), Solidity 0.8.28, OpenZeppelin 5.6.1
MQTT	Mosquitto 2.1.2, nghe cổng 1883 trên mạng Wi-Fi
Mạng	Hotspot điện thoại 2,4 GHz; laptop và ESP32 cùng mạng
Firmware	PlatformIO, Arduino core 2.0.17 cho ESP32
AI	scikit-learn 1.9.1, TensorFlow 2.21 (chỉ khi huấn luyện và lượng tử hóa)
Giao diện	React 19, ethers 6.17, MetaMask

4.2 Kết quả IoT

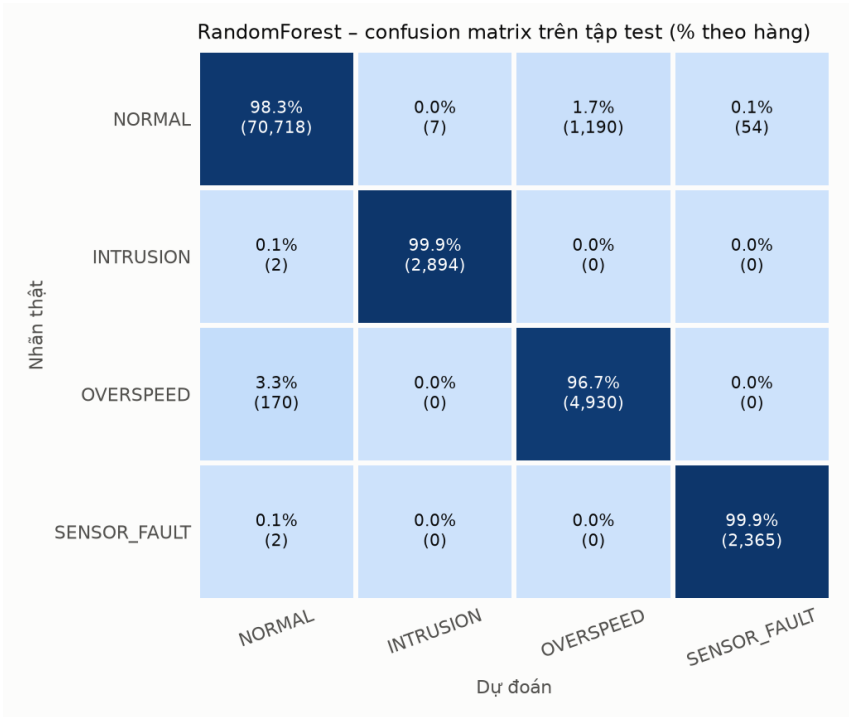
Bảng 4.2: Kết quả tầng IoT

Hạng mục	Kết quả
Logic firmware so với simulator	Biên dịch logic C++ bằng g++ và so từng chu kỳ với simulator: trùng khớp trên 5 kịch bản
Quy đổi HC-SR04	Kiểm tra biên tầm đo, lỗi cảm biến, ngoài tầm; cố ý cài 4 lỗi vào code thì cả 4 đều bị phát hiện
Mạch thật - khởi động	Tự kiểm tra HMAC và Edge AI đạt; HC-SR04 phản hồi
Mạch thật - phân vùng	Tay ở khoảng 48 cm: SAFE, servo quay; khoảng 3,6 cm khi robot đứng yên: WARNING; dưới 5 cm khi robot chạy: DANGER, relay ngắt
Mạch thật - kết nối	ESP32 vào hotspot, kết nối Mosquitto, gửi telemetry 10 Hz sau khi đồng bộ giờ NTP (khoảng 20 giây sau khi cấp điện)

4.3 Kết quả AI

Bảng 4.3: Kết quả phân loại của Random Forest (không tính lỗi trôi dạt)

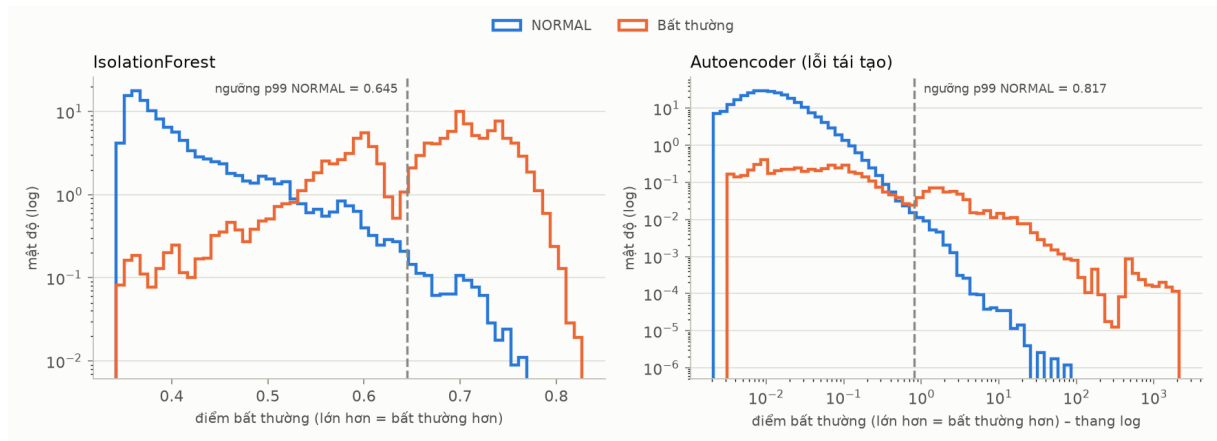
Nhãn	Precision	Recall	F1	Số mẫu
NORMAL	0,998	0,983	0,990	71.969
INTRUSION	0,998	0,999	0,998	2.896
OVERSPEED	0,806	0,967	0,879	5.100
SENSOR_FAULT	0,978	0,999	0,988	2.367
Accuracy			0,983	82.332



Hình 4.1: Ma trận nhầm lẫn của Random Forest

Bảng 4.4: Kết quả phát hiện bất thường

Mô hình	Precision	Recall	F1	ROC-AUC	Báo nhầm
Isolation Forest	0,974	0,662	0,788	0,978	0,8%
Autoencoder	0,977	0,929	0,952	0,989	1,0%
Kết hợp IF + AE	0,969	0,958	0,963	0,993	1,4%



Hình 4.2: Phân bố điểm bất thường của dữ liệu bình thường và bất thường

Kết quả quan trọng nhất là khả năng xác nhận sự cố trước khi ghi lên blockchain (Bảng 4.5). Trong 2.544 cửa sổ vùng DANGER của tập test có 2.390 cửa sổ là người thật, phần còn lại do cảm biến báo sai.

Bảng 4.5: Xác nhận sự cố vùng DANGER

Cách	Precision	Recall	F1	Sự cố giả
Luật: cứ DANGER là sự cố	0,939	1,000	0,969	154
AI: Random Forest xác nhận INTRUSION	1,000	1,000	1,000	0

Nếu chỉ dùng luật, robot sẽ bị khóa oan 154 lần và mỗi lần phải gọi supervisor. Dùng AI thì không có lần khóa oan nào mà vẫn không bỏ sót người thật. Thời gian suy luận trung bình trên backend là 9,7 ms (p95 là 10,8 ms).

4.4 Kết quả Smart Contract

Contract có 50 test: 45 test kịch bản (mocha) và 5 fuzz test (mỗi test chạy 256 lần với đầu vào ngẫu nhiên). Tất cả đều đạt và độ phủ dòng lệnh là 100%. Các test kiểm tra cả trường hợp đúng lẫn trường hợp phải bị từ chối, ví dụ operator, device và cả admin gọi `unlock` đều bị từ chối, gửi lại batch cũ bị từ chối, sửa một trường của bản ghi thì `verifyRecord` trả về `false`.

Bảng 4.6: Chi phí gas (solc 0.8.28, optimizer 200 runs)

Hàm	Gas
Deploy contract	1.623.828
anchorLog (batch 256 bản ghi)	84.540
reportIncident	122.235
unlock	33.279
checkIn	33.094
recordAudit	29.896
verifyRecord (view, proof 8 phần tử)	35.020

Có thể thấy `anchorLog` cho 256 bản ghi (84.540 gas) gần bằng cho 1 bản ghi (84.552 gas), vì trên chuỗi chỉ lưu một root. Bảng 4.7 so sánh chi phí một ca 8 giờ (10 bản ghi/giây, giá gas giả định 1 gwei).

Bảng 4.7: Chi phí một ca 8 giờ

Cách ghi	Số giao dịch	Gas	Chi phí
Neo theo batch 256 bản ghi	1.125	95.107.500	$\approx 0,095$ ETH
Ghi từng dòng	288.000	24.350.976.000	$\approx 24,35$ ETH

Ghi theo batch rẻ hơn khoảng 256 lần. Ghi từng dòng còn cần 10 giao dịch mỗi giây, trong khi Sepolia 12 giây mới có một khối, nên không khả thi.

4.5 Kết quả end-to-end

Bảng 4.8: Kết quả demo trên mạch thật

Màn	Thao tác	Kết quả quan sát
1	Để trống trước cảm biến	Servo quay, dashboard SAFE, cứ 8 giây có một batch được neo lên chuỗi
2	Đưa tay vào dưới 5 cm	Servo dừng ngay, còi kêu; khoảng 1 giây sau dashboard chuyển LOCKED và có dòng sự cố mới; rút tay ra robot vẫn đứng yên
3	Ví OPERATOR gọi unlock	Contract từ chối (<code>AccessControlUnauthorizedAccount</code>); 10 giây sau robot vẫn LOCKED, relay vẫn ngắt
4	SUPERVISOR ký unlock bằng MetaMask	Giao dịch được ghi lên chuỗi; dashboard ACTIVE, relay đóng, servo quay lại
5	Sửa một dòng log trong SQLite	Root tính lại lệch với root trên chuỗi; HMAC chỉ ra đúng dòng bị sửa

Khi chạy lại hệ thống, nhóm còn thấy cơ chế tự đối chiếu hoạt động: ESP32 vẫn nhớ trạng thái LOCKED từ buổi trước, nhưng trên chuỗi mới robot đang ACTIVE, nên backend tự gửi UNLOCK có ký để hai bên khớp nhau. Trên chuỗi local, thời gian từ lúc vào DANGER tới lúc gửi lệnh LOCK là khoảng 0,3-1,1 giây; robot đã dừng từ trước đó nhờ fast lane. Trên Sepolia, thời gian này sẽ tăng lên khoảng 15-30 giây do thời gian tạo khối, nhưng không ảnh hưởng tới việc dừng robot.

CHƯƠNG 5

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết luận

Đề tài đã xây dựng thành công mô hình hệ thống “hộp đen” an toàn cho robot hợp tác, tích hợp đủ ba tầng IoT, AI và Blockchain. Hệ thống chạy thông suốt từ lúc thu thập dữ liệu cảm biến đến khi ghi nhận sự cố trên Smart Contract. Các kết quả chính đạt được gồm:

- Tầng IoT dùng ESP32 và cảm biến HC-SR04 để tính khoảng cách an toàn theo nguyên lý SSM và tự dừng robot ngay tại chỗ, không phụ thuộc vào mạng.
- Tầng AI dùng Isolation Forest phát hiện được các trường hợp bất thường như robot vượt tốc hoặc cảm biến nhiễu, hỗ trợ xác nhận sự cố.
- Smart Contract trên Hardhat thực hiện được việc công chứng nhật ký bằng Merkle root, tự động khóa robot khi có sự cố, và chỉ cho phép người giám sát mở khóa. Mọi thao tác đều được lưu vết.
- Hệ thống phát hiện chính xác trường hợp nhật ký bị chỉnh sửa, cho thấy tính bất biến của dữ liệu được đảm bảo.

Qua đề tài, nhóm hiểu rõ hơn cách kết hợp các công nghệ khác nhau trong một hệ thống thực - ảo. Nhóm cũng nhận ra blockchain nên dùng để truy vết và phân định trách nhiệm, không nên đưa vào vòng điều khiển an toàn đòi hỏi phản hồi nhanh.

5.2 Hướng phát triển

- Thay HC-SR04 bằng cảm biến chính xác hơn như ToF, LiDAR hoặc camera, để giảm sai số và mở rộng vùng giám sát.
- Tích hợp chip bảo mật (secure element) hoặc TEE, để ESP32 tự ký dữ liệu và giao dịch, hạn chế nguy cơ giả mạo dữ liệu đầu vào.
- Triển khai Smart Contract lên mạng thử nghiệm công khai hoặc blockchain riêng cho nhà máy, và áp dụng cơ chế đa chữ ký (multisig) cho thao tác mở khóa.
- Kết nối với robot thật hoặc môi trường mô phỏng ROS 2 để đánh giá hệ thống trong điều kiện gần với thực tế hơn.

Tài liệu tham khảo

- [1] NIST, “Framework for cyber-physical systems: Volume 1, overview,” National Institute of Standards and Technology, NIST Special Publication 1500-201, 2017.
- [2] ISO, “ISO 10218-1:2011, robots and robotic devices – safety requirements for industrial robots – part 1: Robots,” 2011.
- [3] —, “ISO/TS 15066:2016, robots and robotic devices – collaborative robots,” 2016.
- [4] J. A. Marvel and R. Norcross, “Implementing speed and separation monitoring in collaborative robot workcells,” *Robotics and Computer-Integrated Manufacturing*, vol. 44, pp. 144–155, 2017.
- [5] S. Nakamoto, “Bitcoin: A peer-to-peer electronic cash system,” 2008.
- [6] V. Buterin, “Ethereum white paper: A next-generation smart contract and decentralized application platform,” 2014.
- [7] R. C. Merkle, “A digital signature based on a conventional encryption function,” in *Advances in Cryptology – CRYPTO ’87*, ser. LNCS, vol. 293, 1988, pp. 369–378.
- [8] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation forest,” in *Proc. IEEE International Conference on Data Mining (ICDM)*, 2008, pp. 413–422.
- [9] OASIS, “MQTT version 5.0,” 2019, oASIS Standard.
- [10] H. Krawczyk, M. Bellare, and R. Canetti, “HMAC: Keyed-hashing for message authentication,” 1997, rFC 2104.
- [11] R. White, G. Caiazza, A. Cortesi, Y. I. Cho, and H. I. Christensen, “Black block recorder: Immutable black box logging for robots via blockchain,” *IEEE Robotics and Automation Letters*, 2019.